VASP Fraud Detection: GraphSAGE Implementation and RLHF System Design

Approach Summary, Challenges, and Final Metrics

Summary of Approach

This project implements a GraphSAGE-based Graph Neural Network for fraud detection in VASP networks. The system classifies nodes into three risk levels: Low, Medium, and High risk, based on both node features and graph topology.

Architecture Design:

- **Model**: 3-layer GraphSAGE with 128 hidden dimensions
- **Components**: Batch normalization, dropout (0.5), weighted CrossEntropyLoss
- **Features**: Node features include transaction volume, degree centrality, and clustering coefficient
- **Edge Features**: Transaction amount, timestamp, and fraud ring indicators
- **Dataset**: 3,000 nodes with 75,000 edges representing financial transactions

Training Strategy:

- Stratified train/validation/test splits (60%/20%/20%) maintaining class distribution
- Class-weighted loss function to handle imbalance (Low: 68.8%, Medium: 18.1%, High: 13.1%)
- Adam optimizer with learning rate scheduling and early stopping (patience=25)
- Feature normalization applied to both node and edge attributes

Key Challenges

1. Class Imbalance: The dataset exhibits severe class imbalance with low-risk nodes dominating (68.8%). Despite weighted loss functions, the medium-risk class remained challenging to predict accurately.

2. Feature Engineering Limitations: The relatively simple feature set (transaction volume, degree, clustering) may not capture complex fraud patterns. More sophisticated graph-level features could improve performance.

3. Graph Topology Complexity: VASP networks exhibit complex patterns where legitimate high-volume transactions can be structurally similar to fraud rings, creating classification ambiguity.

Final Metrics

Overall Performance:

- **Test Accuracy**: 58%
- **Macro F1-Score**: 0.645
- **Macro ROC-AUC**: 0.775 (Target: >0.85)

Per-Class Performance:

- **Low Risk (Class 0)**: Precision=0.80, Recall=0.52, F1=0.63, AUC=0.726
- **Medium Risk (Class 1)**: Precision=0.22, Recall=0.52, F1=0.31, AUC=0.600

- **High Risk (Class 2)**: Precision=1.00, Recall=0.99, F1=0.99, AUC=1.000

The model excels at identifying high-risk nodes but struggles with medium or low risk classification.

Reinforcement Learning with Human Feedback (RLHF) System Design

1. Feedback Mechanism

Analyst Interface Design: The feedback system would implement a web-based interface where fraud analysts can review model predictions and provide corrections. The interface would display:

- **Node Visualization**: Interactive graph showing the target node and its k-hop neighborhood
- **Prediction Confidence**: Model's probability distribution across risk levels
- **Feature Importance**: Which node/edge features contributed most to the prediction
- **Historical Context**: Previous analyst decisions on similar node patterns

Feedback Collection:

- **Binary Feedback**: Correct/Incorrect classification with optional confidence scores
- **Ranking Feedback**: Analysts rank multiple nodes by risk level for preference learning
- **Explanation Feedback**: Analysts highlight which graph patterns influenced their decision
- **Temporal Feedback**: Corrections based on time-delayed ground truth (actual fraud outcomes)

2. Reward Model

Objective: The reward model aims to predict analyst approval probability for GNN predictions, learning to distinguish between high-quality and low-quality risk assessments from the analyst's perspective.

Architecture:

- **Input**: Node embeddings from the GNN, prediction probabilities, graph structure features, and analyst historical preferences
- **Output**: Scalar reward score indicating expected analyst satisfaction
- **Training Data**: Collected analyst feedback pairs (prediction, approval/rejection)

Training Strategy: The reward model would be trained using preference learning on analyst feedback pairs. For each node, we collect multiple GNN predictions and analyst rankings, then train the reward model to predict relative preferences using a pairwise ranking loss.

3. Refinement Strategy

Algorithm Choice: Direct Preference Optimization (DPO)

Justification: DPO is selected over PPO for several key reasons:

- **Stability**: DPO directly optimizes the GNN without requiring a separate value function, reducing training instability common in GNN applications
- **Sample Efficiency**: More efficient use of limited analyst feedback, crucial given the high cost of expert annotations
- **Implicit Reward Learning**: DPO learns preferences directly from analyst comparisons without explicitly modeling reward signals

- **Graph-Friendly**: Better suited for discrete graph structures compared to continuous control methods like PPO

4. Key Challenge

Primary Challenge: Maintaining Graph Structure Coherence

The fundamental difficulty lies in preserving meaningful graph representations while incorporating human feedback. Unlike text or image domains, graph modifications can break topological relationships critical for fraud detection.

Specific Issues:

- **Non-Local Dependencies**: Changes to one node's classification can affect distant nodes through graph propagation
- **Structural Bias**: Human analysts may focus on local patterns while missing graph-wide fraud rings
- **Feature Drift**: Updating node embeddings based on feedback might disrupt learned structural representations
- **Temporal Consistency**: Maintaining coherent risk assessments as the graph evolves over time

5. Integration with GraphSAGE Architecture

Integration Strategy:

Phase 1 - Embedding Refinement: The RLHF system would fine-tune the GraphSAGE model's final classification layer while keeping the early convolution layers frozen. This preserves learned structural representations while adapting decision boundaries based on analyst feedback.

Phase 2 - Selective Attention: Implement an attention mechanism that weighs different neighborhood aggregations based on analyst preferences. The SAGEConv layers would be augmented with learnable attention weights informed by the reward model.

Phase 3 - Hierarchical Refinement: Apply DPO at multiple levels:

- **Node-level**: Direct classification refinement
- **Subgraph-level**: Learning from analyst feedback on connected components
- **Graph-level**: Incorporating global fraud pattern preferences

Technical Implementation: The refined GraphSAGE would maintain the original 3-layer architecture but add preference-aware components:

- Attention-weighted neighbor aggregation based on analyst feedback patterns
- Confidence-calibrated output layers that adjust prediction certainty based on reward model scores
- Regularization terms that prevent deviation from structural learning in unexplored graph regions

This integration ensures that human expertise enhances rather than replaces the model's ability to detect complex graph-based fraud patterns.